

```

1  """
2  =====
3  Data Cleaning & Insights / Trends Analysis Script
4  Dataset : Raw Data.xlsx (5,000 rows × 31 columns)
5  =====
6
7  USAGE
8  ----
9      python data_analysis.py
10
11  OUTPUT
12  -----
13      cleaned_data.csv          - fully cleaned dataset
14      insights_report.txt       - text summary of key findings
15      plots/                    - folder with all saved charts
16  """
17
18  # -----
19  # 0.  IMPORTS
20  # -----
21  import os
22  import warnings
23  import textwrap
24
25  import numpy as np
26  import pandas as pd
27  import matplotlib
28  matplotlib.use("Agg") # non-interactive backend (no GUI needed)
29  import matplotlib.pyplot as plt
30  import matplotlib.ticker as mticker
31  import seaborn as sns
32  from scipy import stats
33
34  warnings.filterwarnings("ignore")
35  sns.set_theme(style="whitegrid", palette="muted", font_scale=1.05)
36
37  # -----
38  # 1.  CONFIGURATION – edit paths here if needed
39  # -----
40  INPUT_FILE = "Raw Data.xlsx"
41  OUTPUT_CSV = "cleaned_data.csv"
42  REPORT_FILE = "insights_report.txt"
43  PLOTS_DIR = "plots"
44  os.makedirs(PLOTS_DIR, exist_ok=True)
45
46  # -----
47  # 2.  HELPER UTILITIES
48  # -----
49  _report_lines: list[str] = []
50
51  def report(text: str = "") -> None:
52      """Print to console AND accumulate for the text report."""
53      print(text)
54      _report_lines.append(text)
55
56  def save_fig(name: str) -> None:
57      path = os.path.join(PLOTS_DIR, f"{name}.png")
58      plt.savefig(path, dpi=150, bbox_inches="tight")
59      plt.close()
60      print(f"    [saved] {path}")
61
62  def section(title: str) -> None:
63      bar = "=" * 60
64      report(f"\n{bar}\n    {title}\n{bar}")
65
66  # -----
67  # 3.  LOAD DATA

```

```

68 # -----
69 section("3. LOADING RAW DATA")
70 df_raw = pd.read_excel(INPUT_FILE, sheet_name="Sheet1")
71 report(f"Raw shape : {df_raw.shape[0]:,} rows × {df_raw.shape[1]} columns")
72 report(f"Columns : {list(df_raw.columns)}")
73
74 # -----
75 # 4. DATA CLEANING
76 # -----
77 section("4. DATA CLEANING")
78 df = df_raw.copy()
79
80 # — 4a. Duplicates -----
81 n_dups = df.duplicated().sum()
82 report(f"Duplicate rows found : {n_dups}")
83 df = df.drop_duplicates()
84
85 # — 4b. Missing-value summary -----
86 miss = df.isnull().sum()
87 miss = miss[miss > 0].sort_values(ascending=False)
88 report("\nMissing values (columns with gaps):")
89 for col, cnt in miss.items():
90     pct = cnt / len(df) * 100
91     report(f"    {col:<42} {cnt:>5} ({pct:.1f}%)")
92
93 # — 4c. Imputation strategy -----
94 # Numeric → median imputation
95 # Categorical → mode imputation (label "Unknown" if mode unclear)
96 report("\nImputation:")
97
98 numeric_missing = [c for c in miss.index if pd.api.types.is_numeric_dtype(df[c])]
99 for col in numeric_missing:
100     med = df[col].median()
101     df[col].fillna(med, inplace=True)
102     report(f"    [{col}] filled {miss[col]} NaN with median = {med:.2f}")
103
104 categorical_missing = [c for c in miss.index if df[c].dtype == object]
105 for col in categorical_missing:
106     mode_val = df[col].mode(dropna=True)
107     fill = mode_val.iloc[0] if len(mode_val) > 0 else "Unknown"
108     df[col].fillna(fill, inplace=True)
109     report(f"    [{col}] filled {miss[col]} NaN with mode = '{fill}'")
110
111 report(f"\nRemaining nulls after imputation : {df.isnull().sum().sum()}")
112
113 # — 4d. Data-type fixes -----
114 df["Number_of_Virtual_Meetings"] = df["Number_of_Virtual_Meetings"].fillna(df[
    "Number_of_Virtual_Meetings"].median()).astype(int)
115
116 # — 4e. Standardise string columns -----
117 str_cols = df.select_dtypes("object").columns.tolist()
118 for col in str_cols:
119     df[col] = df[col].astype(str).str.strip().str.title()
120
121 # — 4f. Outlier detection & capping (IQR method on numeric cols) -----
122 report("\nOutlier capping (IQR ×1.5 whiskers):")
123 numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
124 # Exclude ID-like cols
125 numeric_cols = [c for c in numeric_cols if c != "Employee_ID"]
126
127 outlier_summary = {}
128 for col in numeric_cols:
129     Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
130     IQR = Q3 - Q1
131     lo, hi = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
132     n_out = ((df[col] < lo) | (df[col] > hi)).sum()
133     if n_out > 0:

```

```

134         df[col] = df[col].clip(lo, hi)
135         outlier_summary[col] = n_out
136         report(f"    {col:<40} {n_out:>4} outliers capped  [{lo:.2f}, {hi:.2f}]]")
137
138 # — 4g. Save cleaned CSV —————
139 df.to_csv(OUTPUT_CSV, index=False)
140 report(f"\nCleaned data saved → {OUTPUT_CSV}  (shape: {df.shape})")
141
142 # —————
143 # 5.  DESCRIPTIVE STATISTICS
144 # —————
145 section("5. DESCRIPTIVE STATISTICS")
146 desc = df[numeric_cols].describe().T[["mean", "std", "min", "50%", "max"]]
147 desc.columns = ["Mean", "Std", "Min", "Median", "Max"]
148 report(desc.round(2).to_string())
149
150 # —————
151 # 6.  CORRELATION ANALYSIS
152 # —————
153 section("6. CORRELATION ANALYSIS")
154 corr = df[numeric_cols].corr()
155
156 # Top 10 strongest correlations (excluding self)
157 corr_pairs = (
158     corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
159     .stack()
160     .reset_index()
161 )
162 corr_pairs.columns = ["Feature_A", "Feature_B", "Correlation"]
163 corr_pairs["Abs"] = corr_pairs["Correlation"].abs()
164 top_corr = corr_pairs.sort_values("Abs", ascending=False).head(10)
165 report("Top 10 feature correlations:")
166 report(top_corr[["Feature_A", "Feature_B", "Correlation"]].to_string(index=False))
167
168 # Plot heatmap
169 fig, ax = plt.subplots(figsize=(14, 11))
170 sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdYlGn", center=0,
171             linewidths=0.4, ax=ax)
172 ax.set_title("Correlation Heatmap – Numeric Features", fontsize=14, pad=12)
173 plt.tight_layout()
174 save_fig("01_correlation_heatmap")
175
176 # —————
177 # 7.  WORKFORCE DEMOGRAPHICS
178 # —————
179 section("7. WORKFORCE DEMOGRAPHICS")
180
181 fig, axes = plt.subplots(1, 3, figsize=(17, 5))
182
183 # 7a. Gender distribution
184 gender_counts = df["Gender"].value_counts()
185 axes[0].pie(gender_counts, labels=gender_counts.index, autopct="%1.1f%%",
186             startangle=140, colors=sns.color_palette("pastel"))
187 axes[0].set_title("Gender Distribution")
188
189 # 7b. Age histogram
190 axes[1].hist(df["Age"], bins=20, color="#5B9BD5", edgecolor="white")
191 axes[1].set_xlabel("Age")
192 axes[1].set_ylabel("Count")
193 axes[1].set_title("Age Distribution")
194
195 # 7c. Years of experience
196 axes[2].hist(df["Years_of_Experience"], bins=20, color="#ED7D31", edgecolor="white")
197 axes[2].set_xlabel("Years of Experience")
198 axes[2].set_ylabel("Count")
199 axes[2].set_title("Experience Distribution")
200

```

```

201 plt.tight_layout()
202 save_fig("02_demographics")
203
204 for g, cnt in gender_counts.items():
205     report(f"    {g:<15} {cnt:>5} ({cnt/len(df)*100:.1f}%)"
206 report(f"    Avg Age                : {df['Age'].mean():.1f} yrs")
207 report(f"    Avg Years Experience: {df['Years_of_Experience'].mean():.1f} yrs")
208
209 # -----
210 # 8. WORK-LOCATION & REMOTE-WORK ANALYSIS
211 # -----
212 section("8. WORK-LOCATION & REMOTE-WORK ANALYSIS")
213
214 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
215
216 # 8a. Work-location breakdown
217 wl = df["Work_Location"].value_counts()
218 axes[0].bar(wl.index, wl.values, color=sns.color_palette("Set2", len(wl)))
219 axes[0].set_title("Work Location Distribution")
220 axes[0].set_ylabel("Employees")
221 for i, v in enumerate(wl.values):
222     axes[0].text(i, v + 20, str(v), ha="center", fontsize=9)
223
224 # 8b. Satisfaction with remote work by location
225 sat_map = {"Unsatisfied": 1, "Neutral": 2, "Satisfied": 3}
226 df["Sat_Score"] = df["Satisfaction with Remote Work"].map(sat_map).fillna(2)
227 sat_by_loc = df.groupby("Work_Location")["Sat_Score"].mean().sort_values(ascending=False)
228 axes[1].barh(sat_by_loc.index, sat_by_loc.values, color=sns.color_palette("coolwarm", len(sat_by_loc)))
229 axes[1].set_xlabel("Avg Satisfaction Score (1=Low, 3=High)")
230 axes[1].set_title("Remote-Work Satisfaction by Work Location")
231
232 plt.tight_layout()
233 save_fig("03_work_location")
234 report(wl.to_string())
235 report(f"\nSatisfaction scores by work location:\n{sat_by_loc.round(2).to_string()}")
236
237 # -----
238 # 9. MENTAL HEALTH & STRESS ANALYSIS
239 # -----
240 section("9. MENTAL HEALTH & STRESS ANALYSIS")
241
242 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
243
244 # 9a. Mental health condition distribution
245 mh = df["Mental_Health_Condition"].value_counts()
246 axes[0].barh(mh.index, mh.values, color=sns.color_palette("Paired", len(mh)))
247 axes[0].set_xlabel("Count")
248 axes[0].set_title("Mental Health Conditions")
249
250 # 9b. Stress level distribution
251 sl = df["Stress_Level"].value_counts()
252 axes[1].pie(sl, labels=sl.index, autopct="%1.1f%",
253             colors=["#FF6B6B", "#FFD93D", "#6BCB77"], startangle=90)
254 axes[1].set_title("Stress Level Distribution")
255
256 # 9c. Stress level vs performance
257 stress_perf = df.groupby("Stress_Level")["Performance"].mean().reindex(["Low", "Medium",
258 "High"])
259 axes[2].bar(stress_perf.index, stress_perf.values,
260             color=["#6BCB77", "#FFD93D", "#FF6B6B"])
261 axes[2].set_ylabel("Avg Performance Score")
262 axes[2].set_title("Avg Performance by Stress Level")
263 axes[2].set_ylim(stress_perf.min() * 0.95, stress_perf.max() * 1.05)
264
265 plt.tight_layout()
266 save_fig("04_mental_health_stress")

```

```

266 report(f"Mental health conditions:\n{mh.to_string()}")
267 report(f"\nStress levels:\n{sl.to_string()}")
268 report(f"\nAvg performance by stress:\n{stress_perf.round(2).to_string()}")
269
270 # -----
271 # 10. PRODUCTIVITY & PERFORMANCE
272 # -----
273 section("10. PRODUCTIVITY & PERFORMANCE")
274
275 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
276
277 # 10a. Productivity change distribution
278 pc = df["Productivity_Change"].value_counts()
279 axes[0].bar(pc.index, pc.values, color=sns.color_palette("Set3", len(pc)))
280 axes[0].set_title("Productivity Change")
281 axes[0].set_ylabel("Count")
282
283 # 10b. Performance distribution
284 axes[1].hist(df["Performance"], bins=25, color="#9E75B7", edgecolor="white")
285 axes[1].set_xlabel("Performance Score")
286 axes[1].set_title("Performance Score Distribution")
287
288 # 10c. Performance by Job Role (top 8)
289 top_roles = df["Job_Role"].value_counts().head(8).index
290 perf_role = (df[df["Job_Role"].isin(top_roles)]
291             .groupby("Job_Role")["Performance"].mean()
292             .sort_values(ascending=False))
293 axes[2].barh(perf_role.index, perf_role.values, color=sns.color_palette("husl", len(
294 perf_role)))
295 axes[2].set_xlabel("Avg Performance")
296 axes[2].set_title("Avg Performance by Job Role")
297
298 plt.tight_layout()
299 save_fig("05_productivity_performance")
300 report(f"Productivity change breakdown:\n{pc.to_string()}")
301 report(f"\nAvg performance score : {df['Performance'].mean():.2f}")
302 report(f"Median performance      : {df['Performance'].median():.2f}")
303
304 # -----
305 # 11. ENERGY CONSUMPTION & COST ANALYSIS
306 # -----
307 section("11. ENERGY CONSUMPTION & COST ANALYSIS")
308
309 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
310
311 # 11a. Monthly kWh by Region
312 kwh_region = df.groupby("Region")["Monthly_kWh"].mean().sort_values(ascending=False)
313 axes[0].bar(kwh_region.index, kwh_region.values, color=sns.color_palette("tab10"))
314 axes[0].set_xticklabels(kwh_region.index, rotation=30, ha="right")
315 axes[0].set_title("Avg Monthly kWh by Region")
316 axes[0].set_ylabel("Monthly kWh")
317
318 # 11b. Electricity cost by Season
319 ec_season = df.groupby("Season")["Electricity_Cost_EGP"].mean().sort_values(ascending=
False)
320 axes[1].bar(ec_season.index, ec_season.values, color=sns.color_palette("Spectral", len(
ec_season)))
321 axes[1].set_title("Avg Electricity Cost by Season (EGP)")
322 axes[1].set_ylabel("Cost (EGP)")
323
324 # 11c. Food + Transport cost scatter vs Performance
325 axes[2].scatter(df["Food_Cost"] + df["Transport_Cost"],
326                df["Performance"], alpha=0.25, s=10, color="#2196F3")
327 axes[2].set_xlabel("Food + Transport Cost")
328 axes[2].set_ylabel("Performance")
329 axes[2].set_title("Living Costs vs Performance")

```

```

330 plt.tight_layout()
331 save_fig("06_energy_costs")
332 report(f"Avg Monthly kWh by Region:\n{kwh_region.round(2).to_string()}")
333 report(f"\nAvg Electricity Cost by Season:\n{ec_season.round(2).to_string()}")
334
335 # -----
336 # 12. WORK-LIFE BALANCE & SLEEP QUALITY
337 # -----
338 section("12. WORK-LIFE BALANCE & SLEEP QUALITY")
339
340 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
341
342 # 12a. Work-life balance rating distribution
343 axes[0].hist(df["Work_Life_Balance_Rating"], bins=5,
344             color="#4CAF50", edgecolor="white")
345 axes[0].set_xlabel("Rating (1-5)")
346 axes[0].set_title("Work-Life Balance Rating")
347
348 # 12b. Sleep quality vs performance
349 sq_perf = df.groupby("Sleep_Quality")["Performance"].mean().sort_values(ascending=False)
350 axes[1].bar(sq_perf.index, sq_perf.values, color=sns.color_palette("Blues_d", len(sq_perf)))
351 axes[1].set_title("Avg Performance by Sleep Quality")
352 axes[1].set_ylabel("Avg Performance")
353
354 # 12c. Hours worked vs work-life balance
355 axes[2].scatter(df["Hours_Worked_Per_Week"], df["Work_Life_Balance_Rating"],
356               alpha=0.2, s=10, color="#FF9800")
357 m, b, r, p, _ = stats.linregress(df["Hours_Worked_Per_Week"], df["Work_Life_Balance_Rating"])
358 x_line = np.linspace(df["Hours_Worked_Per_Week"].min(), df["Hours_Worked_Per_Week"].max(), 100)
359 axes[2].plot(x_line, m * x_line + b, "r-", linewidth=2, label=f"r={r:.2f}")
360 axes[2].set_xlabel("Hours Worked / Week")
361 axes[2].set_ylabel("Work-Life Balance Rating")
362 axes[2].set_title("Hours Worked vs Work-Life Balance")
363 axes[2].legend()
364
365 plt.tight_layout()
366 save_fig("07_worklife_sleep")
367 report(f"Avg Performance by Sleep Quality:\n{sq_perf.round(2).to_string()}")
368 report(f"\nCorrelation (Hours Worked ↔ Work-Life Balance): r = {r:.3f} (p={p:.4f})")
369
370 # -----
371 # 13. INDUSTRY & COUNTRY TRENDS
372 # -----
373 section("13. INDUSTRY & COUNTRY TRENDS")
374
375 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
376
377 # 13a. Employee count by Industry
378 ind_cnt = df["Industry"].value_counts()
379 axes[0].barh(ind_cnt.index, ind_cnt.values, color=sns.color_palette("tab20", len(ind_cnt)))
380 axes[0].set_xlabel("Employee Count")
381 axes[0].set_title("Employees by Industry")
382
383 # 13b. Avg Performance by Country (top 10)
384 top_countries = df["Country"].value_counts().head(10).index
385 perf_country = (df[df["Country"].isin(top_countries)]
386               .groupby("Country")["Performance"].mean()
387               .sort_values(ascending=False))
388 axes[1].barh(perf_country.index, perf_country.values,
389             color=sns.color_palette("husl", len(perf_country)))
390 axes[1].set_xlabel("Avg Performance")
391 axes[1].set_title("Avg Performance by Country (Top 10)")
392

```

```

393 plt.tight_layout()
394 save_fig("08_industry_country")
395 report(f"Employee count by Industry:\n{ind_cnt.to_string()}")
396 report(f"\nAvg Performance by Country:\n{perf_country.round(2).to_string()}")
397
398 # -----
399 # 14. KEY INSIGHTS SUMMARY
400 # -----
401 section("14. KEY INSIGHTS SUMMARY")
402
403 insights = []
404
405 # Insight 1 – Stress vs Performance
406 high_stress_perf = df[df["Stress_Level"] == "High"]["Performance"].mean()
407 low_stress_perf = df[df["Stress_Level"] == "Low"]["Performance"].mean()
408 diff = high_stress_perf - low_stress_perf
409 insights.append(
410     f"1. STRESS & PERFORMANCE: Employees with HIGH stress score "
411     f"{high_stress_perf:.2f} vs {low_stress_perf:.2f} for LOW stress "
412     f"({'higher' if diff > 0 else 'lower'} by {abs(diff):.2f} pts). "
413 )
414
415 # Insight 2 – Work location & satisfaction
416 best_sat = sat_by_loc.idxmax()
417 insights.append(
418     f"2. REMOTE SATISFACTION: '{best_sat}' workers report the highest "
419     f"satisfaction score ({sat_by_loc[best_sat]:.2f}/3). "
420 )
421
422 # Insight 3 – Sleep & performance
423 best_sleep = sq_perf.idxmax()
424 insights.append(
425     f"3. SLEEP & PERFORMANCE: '{best_sleep}' sleepers achieve the best "
426     f"avg performance ({sq_perf[best_sleep]:.2f}). "
427 )
428
429 # Insight 4 – Mental health access
430 mh_col = "Access_to_Mental_Health_Resources"
431 mh_access = df.groupby(mh_col)["Performance"].mean()
432 yes_val = mh_access.get("Yes", mh_access.iloc[-1])
433 no_val = mh_access.get("No", mh_access.iloc[0])
434 insights.append(
435     f"4. MENTAL-HEALTH RESOURCES: Access=Yes → avg performance "
436     f"{yes_val:.2f} vs Access=No → {no_val:.2f}. "
437 )
438
439 # Insight 5 – Top region by energy use
440 top_region = kwh_region.idxmax()
441 insights.append(
442     f"5. ENERGY: '{top_region}' has the highest avg monthly consumption "
443     f"({kwh_region[top_region]:.1f} kWh/month). "
444 )
445
446 # Insight 6 – Productivity change majority
447 pc_majority = pc.idxmax()
448 insights.append(
449     f"6. PRODUCTIVITY: The most common self-reported change is "
450     f"'{pc_majority}' ({pc[pc_majority]} employees, "
451     f"{pc[pc_majority]/len(df)*100:.1f}%). "
452 )
453
454 # Insight 7 – Hours worked / balance correlation
455 dir_word = "negative" if r < 0 else "positive"
456 insights.append(
457     f"7. WORK-LIFE BALANCE: {dir_word.capitalize()} correlation (r={r:.3f}) "
458     f"between hours worked and work-life balance rating "
459     f"({'more hours = lower balance' if r < 0 else 'more hours = higher balance'})."

```

```

460 )
461
462 # Insight 8 – Top performing industry
463 top_ind = df.groupby("Industry")["Performance"].mean().idxmax()
464 top_ind_score = df.groupby("Industry")["Performance"].mean().max()
465 insights.append(
466     f"8. TOP INDUSTRY: '{top_ind}' leads in avg performance "
467     f"({top_ind_score:.2f})."
468 )
469
470 for ins in insights:
471     report("\n" + textwrap.fill(ins, width=72))
472
473 # -----
474 # 15.  SAVE TEXT REPORT
475 # -----
476 with open(REPORT_FILE, "w", encoding="utf-8") as f:
477     f.write("\n".join(_report_lines))
478
479 print(f"\n{'='*60}")
480 print(f"  ALL DONE!")
481 print(f"{'='*60}")
482 print(f"  Cleaned data   → {OUTPUT_CSV}")
483 print(f"  Text report    → {REPORT_FILE}")
484 print(f"  Charts folder → {PLOTS_DIR}/  ({len(os.listdir(PLOTS_DIR))} files)")
485

```